



Nombre de la práctica	Preparacion del DataSet			No.	7
Asignatura:	Simulación	Carrera:	ISIC	Duración de la práctica (Hrs)	2

I. Competencia(s) específica(s): Aprender a realizar la preparacion del DataSet con python

II. Lugar de realización de la práctica (laboratorio, taller, aula u otro): Aula

III. Material empleado:

Computadora

Python

IV. Desarrollo de la práctica:

Preparacion del DataSet

En este Notebook se muestran algunas de las tecnicas mas utilizadas para transformar el DataSet.

DataSet

Abstract

NSL-KDD is a data set suggested to solve some of the inherent problems of the KDD'99 data set which are mentioned in [1]. Although, this new version of the KDD data set still suffers from some of the problems discussed by McHugh [2] and may not be a perfect representative of existing real networks, because of the lack of public data sets for network-based IDSs, we believe it still can be applied as an effective benchmark data set to help researchers compare different intrusion detection methods. Furthermore, the number of records in the NSL-KDD train and test sets are reasonable. This advantage makes it affordable to run the experiments on the complete set without the need to randomly select a small portion. Consequently, evaluation results of different research work will be consistent and comparable.

Data Files

- **KDDTrain+.ARFF** - The full NSL-KDD train set with binary labels in ARFF format
- **KDDTrain+.TXT** - The full NSL-KDD train set including attack-type labels and difficulty level in CSV format
- **KDDTrain+_20Percent.ARFF** - A 20% subset of the KDDTrain+.arff file
- **KDDTrain+_20Percent.TXT** - A 20% subset of the KDDTrain+.txt file
- **KDDTest+.ARFF** - The full NSL-KDD test set with binary labels in ARFF format
- **KDDTest+.TXT** - The full NSL-KDD test set including attack-type labels and difficulty level in CSV format
- **KDDTest-21.ARFF** - A subset of the KDDTest+.arff file which does not include records with difficulty level of 21 out of 21
- **KDDTest-21.TXT** - A subset of the KDDTest+.txt file which does not include records with difficulty level of 21 out of 21

Imports

```
import arff
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
```

Funciones auxiliares

```
def load_kdd_dataset(data_path):
    """Lectura del DataSet NLS-KDD"""
    with open(data_path, 'r') as train_set:
        dataset = arff.load(train_set)
        attributes = [attr[0] for attr in dataset['attributes']]
        return pd.DataFrame(dataset['data'], columns = attributes)
```

```
# shuffle hace el random en una solaparte de la informacion
def train_val_test_split(df, rstate=42, shuffle=True, stratify=None):
    strat = df[stratify] if stratify else None
    train_set, test_set = train_test_split(
        df, test_size=0.4, random_state=rstate, shuffle=shuffle, stratify=strat) # testemos con el 40% y volvemos a revolver
    strat = test_set[stratify] if stratify else None
    val_set, test_set = train_test_split(
        test_set, test_size=0.5, random_state=rstate, shuffle=shuffle, stratify=strat)
    return (train_set, val_set, test_set)
```

1.- Lectura del DataSet

```
df = load_kdd_dataset('datasets/NSL-KDD/KDDTrain+.arff')
```

```
df
```

	duration	protocol_type	service	flag	src_bytes	dst_bytes	land	wrong_fragment	urgent	hot	...	dst_host_srv_count	dst_host_same_srv_rate
0	0.0	tcp	ftp_data	SF	491.0	0.0	0	0.0	0.0	0.0	...	25.0	0.17
1	0.0	udp	other	SF	146.0	0.0	0	0.0	0.0	0.0	...	1.0	0.00
2	0.0	tcp	private	S0	0.0	0.0	0	0.0	0.0	0.0	...	26.0	0.10
3	0.0	tcp	http	SF	232.0	8153.0	0	0.0	0.0	0.0	...	255.0	1.00
4	0.0	tcp	http	SF	199.0	420.0	0	0.0	0.0	0.0	...	255.0	1.00
...	...	...	...	...	...	...	...	...	...	...	...	...	...
125968	0.0	tcp	private	S0	0.0	0.0	0	0.0	0.0	0.0	...	25.0	0.10
125969	8.0	udp	private	SF	105.0	145.0	0	0.0	0.0	0.0	...	244.0	0.96
125970	0.0	tcp	smtp	SF	2231.0	384.0	0	0.0	0.0	0.0	...	30.0	0.12
125971	0.0	tcp	klogin	S0	0.0	0.0	0	0.0	0.0	0.0	...	8.0	0.03
125972	0.0	tcp	ftp_data	SF	151.0	0.0	0	0.0	0.0	0.0	...	77.0	0.30

125973 rows x 42 columns

2.- Division del DataSet

```
train_set, val_set, test_set = train_val_test_split(df, stratify = 'protocol_type')
```

```
print('Longitud del Training Set', len(train_set))
print('Longitud del validation Set', len(val_set))
print('Longitud del test set', len(test_set))
```

```
Longitud del Training Set 75583
Longitud del validation Set 25195
Longitud del test set 25195
```

3.- Limpiando datos

Antes de comenzar es necesario recuperar el DataSet limpio y separa las etiquetas de resto de los datos, no necesariamente aplicar las mismas transformaciones en ambos conjuntos

```
# Separa las características de entrada de las características de salida
X_train = train_set.drop('class', axis = 1)
y_train = train_set['class'].copy()
```



```
# Para ilustrar esta seccion es necesario anadir algunos valores nulos a algunas características del DataSet.
X_train.loc[(X_train['src_bytes']>400) & (X_train['src_bytes']<800), 'src_bytes'] = np.nan
X_train.loc[(X_train['src_bytes']>500) & (X_train['src_bytes']<2000), 'src_bytes'] = np.nan
X_train
```

	duration	protocol_type	service	flag	src_bytes	dst_bytes	land	wrong_fragment	urgent	hot	...	dst_host_count	dst_host_srv_count
113467	0.0	tcp	http	SF	NaN	53508.0	0	0.0	0.0	0.0	...	9.0	255.0
31899	0.0	tcp	private	S0	0.0	0.0	0	0.0	0.0	0.0	...	255.0	4.0
108116	0.0	tcp	http	SF	304.0	636.0	0	0.0	0.0	0.0	...	39.0	255.0
89913	0.0	tcp	private	S0	0.0	0.0	0	0.0	0.0	0.0	...	255.0	15.0
106319	0.0	icmp	eco_i	SF	8.0	0.0	0	0.0	0.0	0.0	...	2.0	7.0
...	...	...	...	...	...	...	...	...	...	...	...	...	...
64559	0.0	tcp	systat	S0	0.0	0.0	0	0.0	0.0	0.0	...	255.0	20.0
67272	0.0	tcp	http	SF	210.0	736.0	0	0.0	0.0	0.0	...	119.0	255.0
32452	3.0	tcp	smtp	SF	NaN	328.0	0	0.0	0.0	0.0	...	111.0	155.0
112657	0.0	tcp	http	SF	284.0	444.0	0	0.0	0.0	0.0	...	255.0	255.0
99030	0.0	tcp	http	SF	209.0	3127.0	0	0.0	0.0	0.0	...	117.0	255.0

75583 rows × 41 columns

La mayoría de los algoritmos de ML no pueden trabajar sobre características que contengan valores nulos por ello existen 3 opciones para reemplazarlos:

- Eliminar las filas correspondientes. Siempre y cuando no haya datos
- Eliminar el atributo (column) correspondiente .
- Rellenarlos con un valor determinado (0, media, mediana, lo que tu quieras).



```
# Comprobar si existe algun atributo con valores nulos
X_train.isna().any()
```

```
duration          False
protocol_type     False
service           False
flag              False
src_bytes         True
dst_bytes         False
land              False
wrong_fragment    False
urgent            False
hot               False
num_failed_logins False
logged_in         False
num_compromised   False
root_shell        False
su_attempted      False
num_root          False
num_file_creations False
num_shells        False
num_access_files  False
num_outbound_cmds False
is_host_login     False
is_guest_login    False
count             False
srv_count         False
```

```
# Seleccionar las filas que contengan valores nulos
filas_valores_nulos = X_train[X_train.isnull().any(axis = 1)]
filas_valores_nulos
```

	duration	protocol_type	service	flag	src_bytes	dst_bytes	land	wrong_fragment	urgent	hot	...	dst_host_count	dst_host_srv_count
113467	0.0	tcp	http	SF	NaN	53508.0	0	0.0	0.0	0.0	...	9.0	255.0
16447	0.0	tcp	smtp	SF	NaN	363.0	0	0.0	0.0	0.0	...	141.0	137.0
64957	1.0	tcp	smtp	SF	NaN	329.0	0	0.0	0.0	0.0	...	198.0	181.0
53498	0.0	tcp	smtp	SF	NaN	330.0	0	0.0	0.0	0.0	...	255.0	108.0
30757	0.0	tcp	ftp_data	SF	NaN	0.0	0	0.0	0.0	0.0	...	188.0	66.0
...	...	...	...	...	...	...	...	...	...	...	...	...	...
124208	0.0	udp	other	SF	NaN	4.0	0	0.0	0.0	0.0	...	255.0	255.0
96968	0.0	tcp	smtp	SF	NaN	337.0	0	0.0	0.0	0.0	...	113.0	205.0
81402	0.0	tcp	ftp_data	SF	NaN	0.0	0	0.0	0.0	0.0	...	137.0	49.0
42166	0.0	tcp	smtp	SF	NaN	328.0	0	0.0	0.0	0.0	...	219.0	139.0
32452	3.0	tcp	smtp	SF	NaN	328.0	0	0.0	0.0	0.0	...	111.0	155.0

6749 rows x 41 columns



Opcion 1: Eliminar las filas con valores nulos

```
# Copiar el DataSet para no alterar el original.
```

```
X_train_copy = X_train.copy()
```

```
#Eliminar las filas con valores nulos.
```

```
X_train_copy.dropna(subset = ["src_bytes", "dst_bytes"], inplace = True)
```

```
X_train_copy
```

	duration	protocol_type	service	flag	src_bytes	dst_bytes	land	wrong_fragment	urgent	hot	...	dst_host_count	dst_host_srv_count
31899	0.0	tcp	private	S0	0.0	0.0	0	0.0	0.0	0.0	...	255.0	4.0
108116	0.0	tcp	http	SF	304.0	636.0	0	0.0	0.0	0.0	...	39.0	255.0
89913	0.0	tcp	private	S0	0.0	0.0	0	0.0	0.0	0.0	...	255.0	15.0
106319	0.0	icmp	eco_i	SF	8.0	0.0	0	0.0	0.0	0.0	...	2.0	7.0
98007	0.0	udp	domain_u	SF	46.0	139.0	0	0.0	0.0	0.0	...	255.0	254.0
...	...	...	...	...	...	...	...	...	...	...	...	...	...
90665	0.0	tcp	ftp_data	S0	0.0	0.0	0	0.0	0.0	0.0	...	255.0	63.0
64559	0.0	tcp	systat	S0	0.0	0.0	0	0.0	0.0	0.0	...	255.0	20.0
67272	0.0	tcp	http	SF	210.0	736.0	0	0.0	0.0	0.0	...	119.0	255.0
112657	0.0	tcp	http	SF	284.0	444.0	0	0.0	0.0	0.0	...	255.0	255.0
99030	0.0	tcp	http	SF	209.0	3127.0	0	0.0	0.0	0.0	...	117.0	255.0

68834 rows x 41 columns

```
# Contar el numero de filas eliminadas
```

```
print('El numero de fiulas eliminadas es: ', len(X_train) - len(X_train_copy))
```

```
El numero de fiulas eliminadas es: 6749
```



Opcion 2: Eliminar los atributos con valores nulos

```
# Copiar el DataSet para no alterar el original
X_train_copy = X_train.copy()
```

```
# Eliminar los atributos con valores nulos
X_train_copy.drop(["src_bytes", "dst_bytes"], axis = 1, inplace = True)
X_train_copy
```

	duration	protocol_type	service	flag	land	wrong_fragment	urgent	hot	num_failed_logins	logged_in	...	dst_host_count	dst_host_srv_count
113467	0.0	tcp	http	SF	0	0.0	0.0	0.0	0.0	1	...	9.0	255.0
31899	0.0	tcp	private	S0	0	0.0	0.0	0.0	0.0	0	...	255.0	4.0
108116	0.0	tcp	http	SF	0	0.0	0.0	0.0	0.0	1	...	39.0	255.0
89913	0.0	tcp	private	S0	0	0.0	0.0	0.0	0.0	0	...	255.0	15.0
106319	0.0	icmp	eco_i	SF	0	0.0	0.0	0.0	0.0	0	...	2.0	7.0
...	...	...	...	...	...	...	...	...	...	...	...	...	...
64559	0.0	tcp	systat	S0	0	0.0	0.0	0.0	0.0	0	...	255.0	20.0
67272	0.0	tcp	http	SF	0	0.0	0.0	0.0	0.0	1	...	119.0	255.0
32452	3.0	tcp	smtp	SF	0	0.0	0.0	0.0	0.0	1	...	111.0	155.0
112657	0.0	tcp	http	SF	0	0.0	0.0	0.0	0.0	1	...	255.0	255.0
99030	0.0	tcp	http	SF	0	0.0	0.0	0.0	0.0	1	...	117.0	255.0

75583 rows x 39 columns

```
# Contar el numero de filas eliminadas
print('El numero de filas eliminadas es: ', len(list(X_train)) - len(list(X_train_copy)))

El numero de filas eliminadas es: 2
```



Opcion 3: Rellenar los valores nulos con un valor determinado

```
# Copiar el DataSet para no alterar el original
X_train_copy = X_train.copy()
```

```
# Rellenar los valores con la media de los valores del atributo
media_srcbytes = X_train_copy["src_bytes"].mean()
media_dstbytes = X_train_copy["dst_bytes"].mean()
X_train_copy.fillna({"src_bytes": media_srcbytes}, inplace = True)
X_train_copy.fillna({"dst_bytes": media_dstbytes}, inplace = True)
X_train_copy
```

	duration	src_bytes	dst_bytes	wrong_fragment	urgent	hot	num_failed_logins	num_compromised	root_shell	su_attempted	...	dst_host_count
0	0.0	18.0	53508.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	9.0
1	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	255.0
2	0.0	304.0	636.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	39.0
3	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	255.0
4	0.0	8.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	2.0
...	...	...	...	...	...	...	...	...	...	...	...	...
75578	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	255.0
75579	0.0	210.0	736.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	119.0
75580	3.0	18.0	328.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	111.0
75581	0.0	284.0	444.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	255.0
75582	0.0	209.0	3127.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	117.0

```
# Copiar el DataSet para no alterar el original
X_train_copy = X_train.copy()
#Rellenar los valores con la mediana
mediana_srcbytes = X_train_copy["src_bytes"].median()
mediana_dstbytes = X_train_copy["dst_bytes"].median()
X_train_copy.fillna({"src_bytes": mediana_srcbytes}, inplace = True)
X_train_copy.fillna({"dst_bytes": mediana_dstbytes}, inplace = True)
X_train_copy
```

	duration	protocol_type	service	flag	src_bytes	dst_bytes	land	wrong_fragment	urgent	hot	...	dst_host_count	dst_host_srv_count
113467	0.0	tcp	http	SF	18.0	53508.0	0	0.0	0.0	0.0	...	9.0	255.0
31899	0.0	tcp	private	S0	0.0	0.0	0	0.0	0.0	0.0	...	255.0	4.0
108116	0.0	tcp	http	SF	304.0	636.0	0	0.0	0.0	0.0	...	39.0	255.0
89913	0.0	tcp	private	S0	0.0	0.0	0	0.0	0.0	0.0	...	255.0	15.0
106319	0.0	icmp	eco_i	SF	8.0	0.0	0	0.0	0.0	0.0	...	2.0	7.0
...	...	...	...	...	...	...	...	...	...	...	...	...	...
64559	0.0	tcp	systat	S0	0.0	0.0	0	0.0	0.0	0.0	...	255.0	20.0
67272	0.0	tcp	http	SF	210.0	736.0	0	0.0	0.0	0.0	...	119.0	255.0
32452	3.0	tcp	smtp	SF	18.0	328.0	0	0.0	0.0	0.0	...	111.0	155.0
112657	0.0	tcp	http	SF	284.0	444.0	0	0.0	0.0	0.0	...	255.0	255.0
99030	0.0	tcp	http	SF	209.0	3127.0	0	0.0	0.0	0.0	...	117.0	255.0

75583 rows × 41 columns



3.1 Imputer Sklearn

Existe otra alternativa para la opción 3, que consiste en usar la clase Imputer de Sklearn

```
X_train_copy = X_train.copy()
```

```
from sklearn.impute import SimpleImputer
imputer = SimpleImputer(strategy = "median")
```

```
# La clase imputer no admite valores categoricos se elimina los atributos categoricos
X_train_copy_num = X_train_copy.select_dtypes(exclude = ['object'])
X_train_copy_num.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 75583 entries, 113467 to 99030
Data columns (total 34 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   duration                             75583 non-null  float64
1   src_bytes                            68834 non-null  float64
2   dst_bytes                            75583 non-null  float64
3   wrong_fragment                       75583 non-null  float64
4   urgent                              75583 non-null  float64
5   hot                                  75583 non-null  float64
6   num_failed_logins                    75583 non-null  float64
7   num_compromised                      75583 non-null  float64
8   root_shell                           75583 non-null  float64
9   su_attempted                        75583 non-null  float64
10  num_root                             75583 non-null  float64
11  num_file_creations                   75583 non-null  float64
12  num_shells                           75583 non-null  float64
13  num_access_files                     75583 non-null  float64
14  num_outbound_cmds                   75583 non-null  float64
15  count                                75583 non-null  float64
16  srv_count                            75583 non-null  float64
17  serror_rate                          75583 non-null  float64
18  srv_serror_rate                      75583 non-null  float64
19  rerror_rate                          75583 non-null  float64
20  srv_rerror_rate                      75583 non-null  float64
21  same_srv_rate                        75583 non-null  float64
22  diff_srv_rate                        75583 non-null  float64
23  srv_diff_host_rate                  75583 non-null  float64
```



```
# Proporcionar atributos numericos para que calcule los valores
imputer.fit(X_train_copy_num)
```

```
SimpleImputer
SimpleImputer(strategy='median')
```

```
# Rellenar los valores nulos
X_train_copy_num_nonan = imputer.transform(X_train_copy_num)

# Trasformar el resultado a un DataFrame de Pandas
X_train_copy = pd.DataFrame(X_train_copy_num_nonan, columns = X_train_copy_num.columns)

X_train_copy.head(10)
```

	duration	src_bytes	dst_bytes	wrong_fragment	urgent	hot	num_failed_logins	num_compromised	root_shell	su_attempted	...	dst_host_count
0	0.0	18.0	53508.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	9.0
1	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	255.0
2	0.0	304.0	636.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	39.0
3	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	255.0
4	0.0	8.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	2.0
5	0.0	46.0	139.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	255.0
6	0.0	18.0	363.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	141.0
7	1.0	18.0	329.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	198.0
8	0.0	206.0	1492.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	255.0
9	0.0	334.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	8.0

10 rows × 34 columns

Conclusión:

La preparación del dataset es un paso crucial antes de entrenar un modelo. En Python, con librerías como pandas, se pueden realizar varias tareas para limpiar los datos. Primero, es común revisar cuántos datos están vacíos en cada columna usando métodos para tener una idea de la cantidad de información faltante. Luego, se pueden eliminar filas o columnas que tengan muchos valores nulos usando `dropna()`. Si los datos faltantes son pocos, una opción es llenarlos con valores como la media o la mediana de la columna usando `fillna()`. También se pueden eliminar columnas que no aporten información relevante o que tengan demasiados datos faltantes. Este proceso ayuda a limpiar el dataset, evitando que los valores vacíos o errores afecten el entrenamiento del modelo y asegurando que los datos sean consistentes y útiles para la fase de modelado.

NOMBRE DEL ALUMNO: Shania Kinnereth Diaz Moya Grupo: 3501 Periodo: 2024-2

RÚBRICA DE EVALUACIÓN ISIC		Puntaje		 Comentarios
Tecnológico de Estudios Superiores de Jilotepec		Máxima	Obtenida	
DOCUMENTO: Prácticas				
Análisis de requerimientos Identifica los datos de relevancia y la problemática asociada.		20		
Diseño / modelado Representación gráfica o escrita de la solución		20		
Codificación / Configuración Traslado del modelado a un lenguaje de programación y/o configuración de la solución.		20		



Pruebas Comparar resultados obtenidos con los esperados	30		
Documentación de la práctica Entrega de manual de prácticas FO-ACA-11	10		
Tiempo de entrega No se califican entregables fuera de tiempo			
Calificación	100		